Cartograph: Verified Executable World Models for ARC-AGI-3

Abstract

Cartograph is a coding agent for ARC-AGI-3 that keeps a persistent, agent-written executable world model and plans inside it only when it passes an exact-replay test: the model must reproduce, cell for cell, every transition in the full recorded history of the current game. While the test fails, the agent continues with open-ended exploration in a Python REPL. The test turns the model into a falsifiable hypothesis and prevents long action sequences from being committed to a model that already has a known counterexample. Cartograph is designed to run entirely locally with an open model on a single GPU and calls no external APIs. We describe the algorithm, explain why the gate works, show that exact replay is informative on all 25 public games, and demonstrate the full loop on real public games in a linked Kaggle notebook.

Introduction

ARC-AGI-3 is an interactive benchmark. An agent is placed in a novel game with no instructions, must infer the rules and the goal from observation, and is scored on how efficiently it acts. Competition rules require offline evaluation, open-source code and a single GPU (https://arcprize.org/competitions/2026).

Two ideas have worked separately: lightweight coding agents that treat a game as an interactive Python problem, and agents that maintain an explicit executable model of the environment. Cartograph combines them with one rule: a world model may influence an action only if it has survived exact replay of everything observed so far.

Prior Work

The Duck harness (Tufa Labs, Milestone 1 winner) exposes game state as Python variables and lets a local open model write and run code to inspect the board and act (https://github.com/Tufalabs/duck-harness, https://tufalabs.ai/research/duck-harness/, https://arcprize.org/blog/arc-prize-2026-milestone-1). Cartograph extends this agent.

Rodionov maintains a persistent executable world model, verifies it against recorded observations and plans through it (https://arxiv.org/abs/2605.05138; code: https://github.com/astroseger/arc-3-agents-baseline1), and a follow-up separates executable models, simplification and exact-replay verification into variants (https://arxiv.org/abs/2607.15439). Kepler validates executable world models with retrospective transition checks and conditional prediction checks and reports a perfect public-set RHAE under a frozen frontier configuration (https://arxiv.org/abs/2610.00834). OPINE-World learns programmatic world models with ontology-error-prioritized interactive exploration (https://arxiv.org/abs/2607.01531).

These systems show that executable models with verification can reach high action efficiency on public games, mostly with frontier APIs. Cartograph differs in setting and mechanism: it inherits Duck's local-model coding loop, requires the replay test to pass before any planning, and falls back to Duck's exploration loop whenever the test fails.

Approach

The agent receives the current frame (letter-coded grid and object segmentation), the action history, and a persistent file `world_model.py`.

1. Model. The agent writes `transition(state, action) -> next_state` in Python. `state` is the grid as a tuple of row strings, one letter per cell; `action` is the recorded action string (for example `UP` or `MOUSE(row=3, col=4)`).
2. Exact replay. After every model edit and every environment action, the verifier replays the whole history from the initial observed state through `transition`. The model is *verified* only if every checked transition equals the observed next frame exactly, including timer and status bars, and at least one transition was checked.
3. Gate. If the model is verified, it is made available to the agent's Python tool and the agent plans inside it (search or rollouts) before acting. If not, the model is unavailable and the agent follows the ordinary explore-and-act loop, together with a report of the first mismatching cells.
4. Act. The chosen action runs in the real game, the observation is appended, and the loop repeats.

```
history ← [initial frame]
loop:
    if replay(model, history) is exact:  action ← plan_inside(model)
    else:                                action ← explore_with_REPL()
    history.append(execute(action))
```

Level changes and `RESET` are discontinuities (a new layout, or a restored start) that a per-level transition function cannot be expected to predict. The replay re-anchors on the observed frame at those steps and reports them separately; they are never counted as passed predictions. The model runs in a resource-limited child process with the same import allowlist as the agent's Python tool, so a verified model is guaranteed to run where it is used for planning.

All components are local; the only persistent state within a game is the model file, the observation log and Duck's own notes and message history.

Theory

Let `D` be the set of (state, action, next state) triples observed so far. Exact replay accepts a model `M` only if `M(s, a) = s'` for every checked triple in `D`; one counterexample rejects it, however plausible its other predictions. The gate has three consequences.

- Plans are correct on everything seen. A plan executed inside a verified model is built from transitions that match reality wherever reality has been observed. A model's remaining errors can only lie on (state, action) pairs not yet observed, and the first such error is detected by the next replay.
- Wasted actions are bounded by detection time. The cost of a wrong model is the actions spent on plans that rely on its unobserved errors. The gate converts a silent error into a detected one at the first counterexample, after which planning stops.
- Revision happens where information is richest. When replay fails, the agent returns to exploration and receives the exact failing cells, which is precisely the evidence needed to repair the model.

This is a minimum-description-length style consistency filter: only models that reproduce the whole record exactly may influence behaviour. The gate does not claim the model is correct on unseen transitions; it guarantees that the model has no known counterexample, and that the cost of any unknown one is detected immediately.

Exact replay is only informative if the environment is deterministic under replay. We measured this directly (next section).

Results

A. Kaggle notebook on public games

The linked notebook runs the Duck solver and the Cartograph agent on real public games in Kaggle's offline environment (CPU only, internet off, 52 seconds). Here the language model's decisions are scripted: a fixed policy writes a model, and the model it writes memorises the transitions seen so far. It therefore verifies when it is written and is out of date one action later. The run shows the whole loop on real engine data: model written, history replayed, planning gate opened and closed, everything logged.

Quantity	Value
Notebook	https://www.kaggle.com/code/datweb3guy/notebook31a12f6226
Games run	3 (g50t, tr87, su15), 16 actions each, 48 in total
Model writes	42; 39 verified; the deliberately wrong model was rejected in 3 of 3 games
Actions issued with the gate open (planning enabled)	39 of 48 (the other 9 before any model existed)
Replays after an action, with a model written	39; 0 verified (each action produced a transition the model had not yet seen)
Levels completed	0 of 22 (the script does not try to win, so this is not an agent result)

B. Exact replay is informative

For each of the 25 public games, three random action sequences of 40 actions (clicks at random coordinates, fixed seed) were played on two fresh environments and compared frame by frame. All 25 games and all 75 sequence replays were identical. The mean number of cells changed per action ranges from 1.1 (tn36) to 112.3 (cn04) across games, so the same test covers games with sparse and dense dynamics.

C. Ablation design

Three variants on the same public games with identical model, temperature and action budget: (1) Duck REPL only; (2) Duck with a persistent model and no forced verification; (3) Duck with a persistent model, exact-replay verification and gated planning. Reported quantities: levels completed, actions to completion, verification success rate.

Universality, Progress, and Novelty

Universality. The pattern (write an executable model, require exact replay of the full history, plan only inside a surviving model) applies to any interactive domain where a coding agent can maintain a transition function and replay is deterministic enough to be informative. Nothing in it is specific to grids.

Progress. Reaching strong ARC-AGI-3 performance with an open, local model requires acting efficiently while rules are still being learned. The gate gives that regime a precise, checkable condition for when a learned model may be trusted.

Novelty. The individual pieces exist in public work. The contribution is the strict gate (planning is disabled unless exact replay passes), its integration into a lightweight local coding loop, and its evaluation under the competition's offline, open-source, single-GPU constraints.

Conclusion

A Duck-style coding agent that must maintain and exactly verify an executable world model before planning turns open-ended exploration into testable hypothesis refinement. The mechanism is small, designed to run locally, and reusable for agents that acquire skills in novel environments with limited interaction.

References

- ARC Prize 2026 rules and paper track: https://arcprize.org/competitions/2026 and https://arcprize.org/competitions/2026/paper
- Tufa Labs, The Duck: https://github.com/Tufalabs/duck-harness and https://tufalabs.ai/research/duck-harness/
- ARC Prize Milestone 1 announcement: https://arcprize.org/blog/arc-prize-2026-milestone-1
- Rodionov, Executable World Models for ARC-AGI-3 in the Era of Coding Agents: https://arxiv.org/abs/2605.05138; follow-up: https://arxiv.org/abs/2607.15439; code: https://github.com/astroseger/arc-3-agents-baseline1
- Wu, Kepler: Auditable World Models for ARC-AGI-3: https://arxiv.org/abs/2610.00834
- Courtis, Li, Sanner, OPINE-World: https://arxiv.org/abs/2607.01531